UNIVERSIDAD DE PAMPLONA · INGENIERÍA DEL SOFTWARE III (167420)

Domain Service de Sistema de Turnos

Guía de funcionamiento, paquetes, integración con AuthCore y uso

Grupo Sistema de Turnos · Historias de usuario HU-8 a HU-12 · Agenda sin cruces

Arquitectura hexagonal · SOLID (Semana 5) · Núcleo de dominio (Semana 6)
Patrones Factory, Repository y Adapter (Semanas 5 a 7)
Java 17 · Spring Boot 3 · Maven · H2 / MySQL · REST y consola

Prof. Carlos Arturo Barrientos Suárez · Octubre de 2026

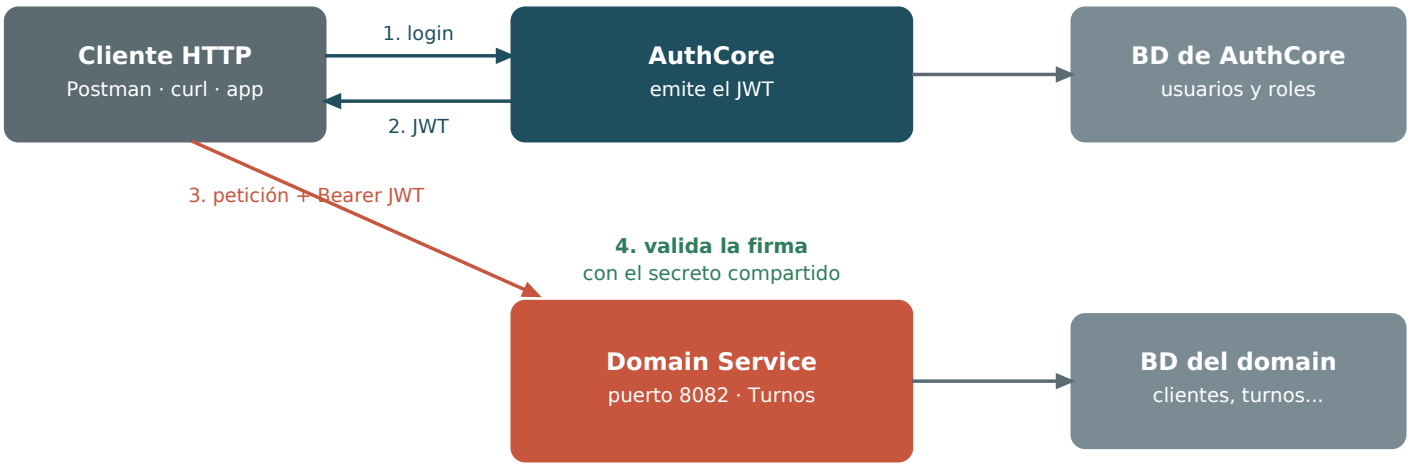
Material de clase: avance del servicio, pensado para leerse con el código al lado.

1. ¿Qué es este servicio y qué resuelve?

El sistema de turnos organiza una agenda: hay **clientes**, hay **profesionales** que atienden en ciertos días y horas, y hay **turnos** (citas) que unen a un cliente con un profesional en un horario. La regla que justifica todo el servicio es una sola: **un profesional no puede tener dos turnos que se crucen**. Este servicio —el **domain service**— contiene esa y las demás reglas. No crea usuarios ni guarda contraseñas: de eso se ocupa **AuthCore**, el servicio que ustedes ya construyeron. El domain service solo *confía en la identidad que AuthCore firma*.

HU	Qué permite	Dónde vive la regla
HU-8	Registrar clientes (nombre, teléfono, correo opcional) y profesionales con su disponibilidad semanal	`Cliente`, `Profesional`, `Franja`
HU-9	Crear un turno (cliente, profesional, fecha, hora y duración; 30 min por defecto)	`Turno.agendar` + `TurnoService`
HU-10	Impedir que dos turnos del mismo profesional se solapen	`RangoTiempo.seCruzaCon` + `ValidadorAgenda`
HU-11	Cancelar o reprogramar un turno	`Turno.cancelar`, `Turno.reprogramar`
HU-12	Consultar turnos por profesional, cliente y rango de fechas	`TurnoRepositoryPort.buscar`
RNF-01	El sistema se puede usar también desde la consola	`TurnosCli` (perfil `cli`)

Mapa de la solución



El domain service NUNCA llama a AuthCore en cada petición.

Idea clave: son dos servicios independientes que comparten un solo secreto. AuthCore firma los tokens con ese secreto; el domain service usa el mismo secreto para comprobar que la firma es auténtica. Nadie más necesita hablar con nadie.

> **Estado de este avance**
> El **núcleo** (dominio, aplicación y consola) está compilado y tiene **66 pruebas automáticas que pasan**. La capa de infraestructura (Spring, JPA, REST, JWT) está escrita junto con sus pruebas, pero todavía **debe validarse en su máquina con** `mvn verify`; si aparece un error ahí, no es un fracaso: es el primer ejercicio real de integración.

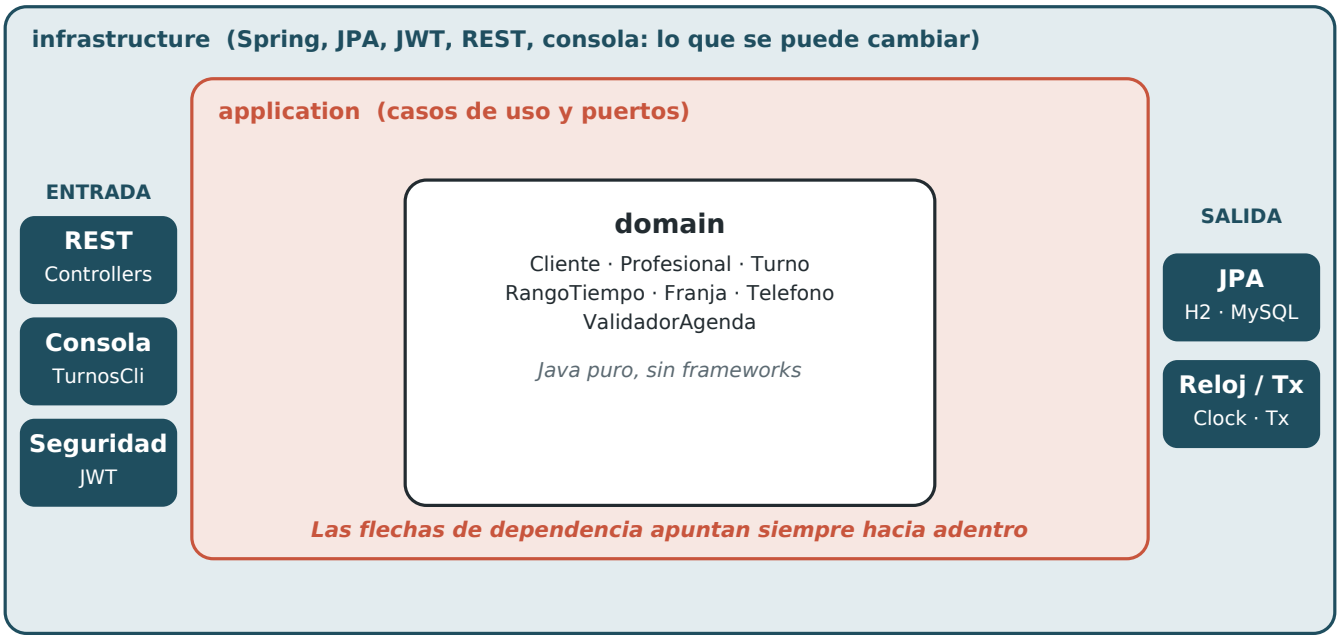
2. Tecnologías utilizadas

El informe del grupo no declaró tecnologías, así que se usó el **estándar del curso**, el mismo de los demás grupos. Si el grupo ya había elegido otras, solo cambia la capa `infrastructure`: el dominio y los casos de uso no se tocan.

Tecnología	Versión	Para qué se usa aquí
Java	17	Lenguaje. Se aprovechan *records* para objetos de valor y comandos inmutables
Spring Boot	3.5.6	Arranca la aplicación y conecta (inyecta) las piezas
Spring Web	—	Controladores REST (`@RestController`) y JSON
Spring Security	6	Cadena de filtros sin sesión (stateless); solo el rol ADMIN opera
jjwt	0.12.6	Verificar la firma y leer los datos del JWT de AuthCore
Spring Data JPA + Hibernate	—	Guardar y leer en base de datos; bloqueo de fila del profesional
H2 / MySQL	—	H2 en memoria para desarrollo y pruebas; MySQL en el perfil `prod`
Maven	3.9+	Compilación, dependencias y pruebas (`mvn verify`)
JUnit 5 + MockMvc	—	Pruebas unitarias y de punta a punta (HTTP → BD)
ArchUnit	1.3.0	Pruebas que vigilan las reglas de la arquitectura

Lo importante: el negocio no depende de ninguna de estas tecnologías

Las clases de `domain` y `application` **no importan una sola clase de Spring, JPA ni jjwt**. Por eso el mismo `TurnoService` atiende a los controladores REST *y* a la consola: son dos puertas de entrada distintas al mismo caso de uso (requisito RNF-01). Eso es *arquitectura hexagonal* y es el principio de inversión de dependencias (la D de SOLID) hecho código.



3. Los paquetes y sus responsabilidades

Cada paquete tiene **una responsabilidad** y una regla sobre a quién puede importar. Si dudan dónde va una clase, pregúntense qué cambia por qué razón.

```
com.sistematurnos.agenda
├── domain                Java puro. Las reglas del negocio.
│   ├── vo/               Telefono · RangoTiempo · Franja
│   ├── model/            Cliente · Profesional · Turno · EstadoTurno
│   ├── service/          ValidadorAgenda (no pasado, no solapado)
│   └── exception/        errores de negocio (TurnoSolapado...)
├── application           Orquesta casos de uso. Solo conoce el dominio.
│   ├── port/in/          lo que el servicio OFRECE (casos de uso + comandos)
│   ├── port/out/         lo que NECESITA (repositorios, transacción)
│   └── service/          ClienteService · ProfesionalService · TurnoService
└── infrastructure        Detalles técnicos. Se puede reemplazar.
    ├── config/           BeanConfig: aquí se arma todo (raíz de composición)
    ├── security/         JwtTokenValidator · JwtAuthenticationFilter · SecurityConfig
    └── adapters/
        ├── in/rest/      Controllers · DTO · RestExceptionHandler
        ├── in/cli/       TurnosCli (consola, mismos casos de uso)
        └── out/          persistence (entity · repository · mapper · *Adapter)
```

Paquete	Responsabilidad	Puede importar	NO puede importar
`domain.vo`	*RangoTiempo* (inicio y fin válidos, mismo día), *Franja* (día y horas de atención), *Telefono* (7 a 15 dígitos normalizados)	solo JDK	todo lo demás
`domain.model`	*Cliente*, *Profesional*, *Turno*. Protegen sus propias reglas (un turno cancelado no se reprograma)	domain.vo, exception	application, infrastructure, frameworks
`domain.service`	*ValidadorAgenda*: reglas que involucran varios turnos (no en el pasado, no cruzado)	domain.*	application, infrastructure
`application.port.in`	El contrato público: «agendar», «cancelar»... más los *comandos* con sus datos	domain	infrastructure
`application.port.out`	Interfaces de lo que se necesita afuera (guardar, buscar, abrir transacción)	domain	infrastructure
`application.service`	Un caso de uso por método: busca, delega al dominio, guarda. Sin lógica de negocio propia	domain, port.*	Spring, JPA, infrastructure
`infrastructure.config`	Construye los objetos y les entrega sus dependencias	todo	—
`infrastructure.security`	Valida el JWT y protege las rutas	Spring Security, jjwt	domain (no lo necesita)
`adapters.in`	REST y consola: traducen la entrada (HTTP o texto) a comandos y los errores de negocio a respuestas	port.in, dto	adapters.out, application.service
`adapters.out`	Implementa los puertos de salida con JPA	port.out, domain, JPA	adapters.in

Estas reglas se prueban solas

La clase `ArquitecturaHexagonalTest` (ArchUnit) falla si alguien hace que el dominio importe Spring o que un controlador llame directamente a la persistencia. La arquitectura deja de ser un dibujo y pasa a ser una regla que el build hace cumplir.

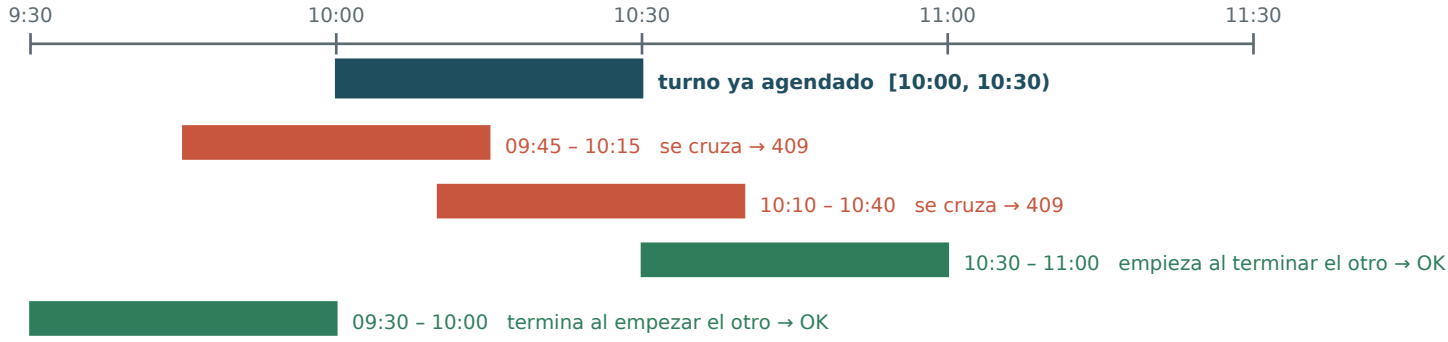
4. Recorrido de una petición: agendar un turno

Seguir una petición completa es la mejor forma de entender para qué sirve cada paquete. Ejemplo: `POST /api/turnos` con un cuerpo como `{clienteId, profesionalId, fecha, hora, duracionMinutos}`, hecho por un ADMIN.

#	Quién	Qué hace
1	Cliente HTTP	Envía el POST con el encabezado `Authorization: Bearer <JWT>` y el JSON del turno
2	`JwtAuthenticationFilter`	Extrae el token y se lo pasa a `JwtTokenValidator`, que comprueba firma y vencimiento. Si es válido, deja en el contexto al usuario y sus roles
3	`SecurityConfig`	Solo el rol ADMIN entra a `/api/**`. Sin token → 401; rol USER → 403
4	`TurnoController`	Convierte el JSON en un `AgendarTurnoCommand` y llama al caso de uso *por su interfaz*
5	`TurnoService`	Valida los campos obligatorios (si faltan varios, los lista todos) y abre una transacción mediante `TransaccionPort`
6	Puertos de salida	Busca el cliente y el profesional (404 si no existen). **Bloquea la fila del profesional** (`SELECT ... FOR UPDATE`) para que dos solicitudes simultáneas se ordenen
7	`RangoTiempo.de()`	Arma el rango con la duración (30 min por defecto, entre 5 y 480, dentro de un mismo día). Inválido → 400
8	`ValidadorAgenda` (dominio)	El turno no puede empezar antes de «ahora» (reloj inyectado). Pasado → 400
9	`Profesional.asegurarQueAtiende`	El rango debe caber dentro de alguna franja de disponibilidad. Si no → 409
10	`ValidadorAgenda.buscarConflicto`	Compara con los turnos AGENDADOS del profesional ese día. Si se cruza → 409 indicando con qué turno
11	`Turno.agendar()`	Crea el turno en estado AGENDADO; se guarda y la transacción se confirma. Responde **201**
✗	`RestExceptionHandler`	Si algo falla, la transacción se revierte y la excepción de negocio se traduce a 400, 404, 409 o 422

El corazón del negocio: cuándo dos turnos se cruzan

Los rangos son **semiabiertos**: incluyen el inicio y excluyen el fin. Por eso un turno de 10:00 a 10:30 y otro que empieza a las 10:30 **no** se cruzan: el profesional puede atender uno detrás del otro sin pausa.



Regla: dos turnos se cruzan si inicioA < finB Y inicioB < finA (intervalos semiabiertos)

Decisiones de diseño para discutir en clase

- **La regla vive en el dominio.** `RangoTiempo.seCruzaCon` no sabe de bases de datos; el puerto de persistencia solo devuelve los turnos agendados del profesional para una fecha y el dominio decide si hay cruce. Se prueba con milisegundos.
- **Dos defensas contra la concurrencia.** El bloqueo de la fila del profesional ordena a quienes agendan a la vez; además `@Version` en las entidades y las restricciones únicas (teléfono, nombre del profesional) protegen en la base de datos.
- **El reloj es una dependencia.** `Clock` se inyecta: las pruebas fijan «hoy» en un martes a las 10:00 y no dependen de cuándo se ejecuten.
- **La transacción es un puerto, no** `@Transactional`**.** Así `application` no importa Spring y el caso de uso se prueba con un doble que ejecuta el código directamente.
- **Cancelar no borra.** El turno pasa a CANCELADO, conserva su historia y libera el horario. Un turno ya pasado no se modifica.

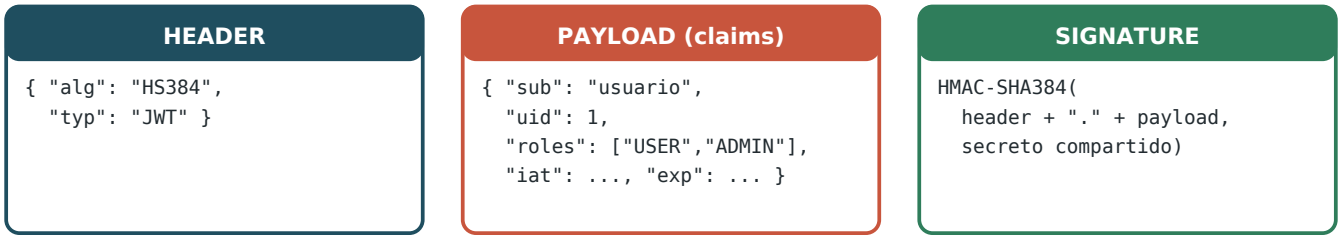
Patrones de las Semanas 5 a 7 en este código

Patrón	Dónde se ve	Para qué
Factory	`Cliente.registrar()`, `Turno.agendar()`, `RangoTiempo.de()`, `reconstruir()`	Constructores con nombre que validan; *registrar/agendar* aplican reglas de negocio, *reconstruir* rehidrata desde la BD
Adapter / Mapper	`TurnoMapper`, `TurnosCli`, los controladores	Traducir entre el mundo exterior (HTTP, texto, tablas) y el modelo de dominio
Repository	`TurnoRepositoryPort` + `TurnoRepositoryAdapter`	El dominio «guarda y busca» sin saber que existe SQL
Domain Service	`ValidadorAgenda`	Una regla que no pertenece a un solo objeto (involucra al nuevo turno y a los existentes)

5. Integración con AuthCore

5.1 Qué recibe y qué verifica el domain service

AuthCore autentica a la persona (usuario y contraseña con BCrypt) y entrega un **JWT**. El domain service nunca ve la contraseña: solo recibe ese token en cada petición y comprueba que sea auténtico.



algoritmo de firma *quién es y hasta cuándo vale* *prueba que nadie lo alteró*

Claim	Ejemplo	Cómo lo usa el domain service
`sub`	`"admin1"`	Identifica al usuario (`UsuarioAutenticado.username`)
`uid`	`1`	Identificador del usuario en AuthCore
`roles`	`["USER","ADMIN"]`	Se convierten en autoridades `ROLE_USER` y `ROLE_ADMIN`. En este servicio solo ADMIN opera
`iat` / `exp`	fechas	Emisión y vencimiento. Un token vencido se rechaza (con 30 s de tolerancia de reloj)

5.2 El secreto compartido

- AuthCore firma con una clave; el domain service necesita **exactamente la misma**, en formato **Base64**.
- Los tokens de AuthCore usan **HS384**, por lo que la clave debe tener **al menos 48 bytes** (384 bits). Una clave más corta falla al arrancar o al validar.
- Se entrega por la variable de entorno `JWT_SECRET_BASE64`. **Nunca** se escribe en el código, ni en `application.yml`, ni en el repositorio. Si la variable no existe, la aplicación **se niega a arrancar**: es intencional.

```
# Linux / macOS / Git Bash
export JWT_SECRET_BASE64='<la misma clave Base64 de AuthCore>'

# Windows PowerShell
$env:JWT_SECRET_BASE64 = '<la misma clave Base64 de AuthCore>'
```

5.3 Quién puede hacer qué

Operación	Sin token	USER	ADMIN
Clientes y profesionales: registrar y consultar	401	403	✓
Turnos: crear, cancelar, reprogramar y consultar	401	403	✓
Cualquier ruta que no sea /api/**	401	403	403

Nota: el informe del grupo no define roles; la propuesta es que **solo el administrador (quien atiende la agenda) opera el sistema**. Si más adelante los clientes agendan solos, habría que añadir permisos para el rol USER y que cada cliente vea únicamente sus turnos. Se cambia en `SecurityConfig`.

5.4 Qué NO hace el domain service (y por qué)

- No llama a AuthCore para validar cada token: sería más lento y haría que, si AuthCore cae, caiga todo. La firma ya prueba la autenticidad.
- No guarda usuarios ni contraseñas. Los clientes de la agenda son *personas atendidas*, no usuarios del sistema.
- No puede revocar un token antes de que venza. Por eso los tokens deben durar poco; la revocación inmediata es una mejora futura.

> **Seguridad: lo que deben cuidar ya**
> La clave compartida, las contraseñas de la base de datos y los tokens son secretos. **No los pongan en documentos, capturas, chats ni en el repositorio.** En esta guía y en `requests.http` se usan variables y marcadores, nunca valores reales. Si alguna vez un secreto se publica, se considera comprometido: se cambia.

6. Cómo usarlo, paso a paso

6.1 Requisitos

- JDK 17 y Maven 3.9 o superior.
- AuthCore corriendo y un usuario con rol **ADMIN**.
- El mismo secreto Base64 con el que AuthCore firma.

6.2 Compilar, probar y ejecutar

```
unzip domain-service-turnos.zip && cd domain-service-turnos
export JWT_SECRET_BASE64='...'      # el secreto compartido

mvn verify                          # compila y corre TODAS las pruebas
mvn spring-boot:run                 # REST en http://localhost:8082 (H2 en memoria)

# con MySQL
export SPRING_PROFILES_ACTIVE=prod
export DB_URL=jdbc:mysql://localhost:3306/turnos
export DB_USER=...  DB_PASSWORD=...       # en variables, no en archivos
mvn spring-boot:run
```

6.3 Obtener el token en AuthCore

Se hace una petición POST al endpoint de login de *su* AuthCore. Los nombres exactos de la ruta y de los campos son los que ustedes definieron; los de abajo son de ejemplo, ajústenlos si difieren.

```
AUTHCORE=http://<host-de-authcore>:<puerto>
DOMAIN=http://localhost:8082

curl -s -X POST $AUTHCORE/api/auth/login -H 'Content-Type: application/json' \
     -d '{"username":"<usuario>","password":"<clave>"}'
TOKEN='eyJhbGciOiJIUzM4NCJ9...'          # copie el token de la respuesta (rol ADMIN)
```

6.4 Probar los casos de uso

```
# HU-8  registrar un cliente y un profesional (atiende lunes 08-12)
curl -s -X POST $DOMAIN/api/clientes -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{"nombre":"Ana Pérez","telefono":"300 123 4567","correo":"ana@correo.com"}'
curl -s -X POST $DOMAIN/api/profesionales -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{"nombre":"Dra. López","disponibilidad":[{"dia":"MONDAY","desde":"08:00","hasta":"12:00"}]}'

# HU-9  crear el turno (un lunes futuro; duración opcional, 30 por defecto)
curl -s -X POST $DOMAIN/api/turnos -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{"clienteId":1,"profesionalId":1,"fecha":"2026-11-02","hora":"09:00"}'

# HU-10 repetir con hora 09:15  ->  409 "se cruza con el turno 1"
# HU-11 reprogramar (solo lo que se envía) y cancelar
curl -s -X PUT $DOMAIN/api/turnos/1/horario -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' -d '{"hora":"10:00"}'
curl -s -X POST $DOMAIN/api/turnos/1/cancelacion -H "Authorization: Bearer $TOKEN"

# HU-12 consultar con filtros (todos opcionales)
curl -s "$DOMAIN/api/turnos?profesionalId=1&desde=2026-11-01&hasta=2026-11-30" \
  -H "Authorization: Bearer $TOKEN"
```

El archivo `requests.http` incluye todas las llamadas listas para IntelliJ o VS Code (extensión REST Client).

6.5 Tabla de endpoints

Método y ruta	Historia	Rol
`POST /api/clientes` · `GET /api/clientes` · `GET /api/clientes/{id}`	HU-8	ADMIN
`POST /api/profesionales` · `GET /api/profesionales` · `GET /api/profesionales/{id}`	HU-8	ADMIN
`POST /api/turnos`	HU-9 · HU-10	ADMIN
`POST /api/turnos/{id}/cancelacion` · `PUT /api/turnos/{id}/horario`	HU-11	ADMIN
`GET /api/turnos?profesionalId&clienteId&desde&hasta&incluirCancelados` · `GET /api/turnos/{id}`	HU-12	ADMIN

6.6 Qué significan las respuestas de error

Código	Cuándo ocurre	Qué revisar
400	Dato inválido: campos obligatorios ausentes (el mensaje los lista todos), teléfono mal formado, duración fuera de 5–480, turno en el pasado, fecha u hora ilegibles, rango de consulta con «desde» posterior a «hasta», JSON mal formado	El cuerpo de la petición
401	Falta el token, está vencido, está alterado o fue firmado con otra clave	Encabezado `Bearer`; que ambos servicios usen el **mismo** secreto; hora del servidor
403	Token válido pero el rol no es ADMIN	El campo `roles` del token
404	Cliente, profesional o turno inexistente	El identificador
409	Teléfono o nombre repetido, profesional que no atiende en ese horario, **turno que se cruza con otro** (el cuerpo indica cuál), turno ya cancelado o pasado, o dos operaciones simultáneas sobre los mismos datos	Leer `detalles` en la respuesta; reintentar si fue concurrencia
422	Otra regla de dominio incumplida	El mensaje de la respuesta
500	Error no previsto. El mensaje es genérico a propósito; el detalle queda en el log	El log del servidor

6.7 Usarlo por consola (RNF-01)

La misma lógica se maneja desde la terminal, sin servidor web ni token (es una herramienta local de administración). Se activa con el perfil `cli`.

```
SPRING_PROFILES_ACTIVE=cli mvn spring-boot:run

> cliente registrar Ana Pérez | 3001234567 | ana@correo.com
> profesional registrar Dra. López | LUNES 08:00-12:00, MIERCOLES 14:00-18:00
> turno agendar ...          (ver «ayuda» para la sintaxis completa)
> turno listar               > turno cancelar 1        > salir
```

7. Reglas de negocio que el código hace cumplir

Varias de estas reglas no aparecen en el informe del grupo; son **decisiones propuestas** para que el servicio sea coherente (están documentadas en el Excel del backlog). Confírmenlas con el cliente: si alguna es distinta, el cambio es pequeño y localizado.

- **Cliente:** nombre y teléfono obligatorios; el teléfono (7 a 15 dígitos, se ignoran espacios y guiones) es único. El correo es opcional y, si viene, debe ser válido.
- **Profesional:** nombre único (sin distinguir mayúsculas) y al menos una franja de disponibilidad; dos franjas del mismo día no pueden traslaparse.
- **Turno:** la duración por defecto es de 30 minutos (entre 5 y 480) y debe terminar el mismo día. Debe caber dentro de la disponibilidad del profesional.
- **No solapamiento:** aplica por profesional; solo cuentan los turnos AGENDADOS. Un mismo cliente puede tener turnos con profesionales distintos a la misma hora.
- **Futuro:** no se agenda ni se reprograma al pasado, y un turno que ya pasó no se modifica.

- **Cancelar:** pasa a CANCELADO y libera el horario. Cancelar dos veces es un conflicto (409).
- **Reprogramar:** cambia fecha, hora o duración (lo que no se envía se conserva) y revalida todo, ignorando el propio turno al buscar cruces.
- **Consultar:** por profesional, cliente y rango de fechas, combinables; ordenado por inicio; excluye cancelados salvo que se pidan. Zona horaria America/Bogota.

Cómo se protege la agenda cuando hay operaciones al mismo tiempo

El riesgo clásico: dos administradores agendan a la misma hora para el mismo profesional en el mismo instante; ambos «ven» el hueco libre y ambos guardan. Para evitarlo, al agendar se **bloquea la fila del profesional** dentro de la transacción: la segunda solicitud espera a que termine la primera y entonces ya ve el turno recién creado, así que recibe el 409. `@Version` en las entidades y las restricciones únicas son defensas adicionales.

> **Limitaciones conocidas**
> El esquema de base de datos se crea con `ddl-auto: update`; antes de producción debe reemplazarse por migraciones (Flyway). Las franjas de disponibilidad son semanales y no contemplan festivos ni vacaciones. El enunciado menciona una API externa, pero no define cuál: no se implementó y se dejó como pregunta abierta para el cliente.

8. Pruebas: cómo sabemos que funciona

Tipo	Dónde	Qué comprueban	Velocidad
Unitarias de dominio	`test/.../domain`	Rangos, franjas, teléfono, cliente, profesional, turno y sus transiciones (cancelar, reprogramar)	milisegundos, sin Spring
De casos de uso	`test/.../application`	Flujos completos con dobles en memoria: orden de validaciones, cruces, cancelados, reprogramar ignorando el propio turno	milisegundos, sin BD
De contrato	`test/.../contract`	Las **mismas** pruebas para el doble en memoria y para el adaptador JPA (principio de Liskov)	rápidas
Consola	`TurnosCliTest`	Los comandos de texto producen los mismos resultados que los controladores, sin mostrar trazas	milisegundos
Arquitectura	`ArquitecturaHexagonalTest`	Que nadie rompa las reglas de dependencia entre paquetes	rápidas
Unicidad	`RestriccionesDeUnicidadTest`	La base de datos rechaza teléfonos y nombres repetidos	rápidas, con H2
Seguridad e integración	`TurnosRestTest`	HTTP real con JWT: 401, 403, flujo completo, cruces 409, cancelar, reprogramar y filtros	segundos, con H2

Estado: las **66 pruebas de dominio, aplicación y consola pasan**. Las de contrato JPA, unicidad y `TurnosRestTest` están escritas pero las debe ejecutar el grupo con `mvn verify`.

Idea para el aula: el reloj se inyecta (`Clock`) y los repositorios son interfaces. Por eso podemos probar «no se agenda al pasado» sin esperar ni tocar una base de datos.

9. Ejercicios para el grupo

Nivel	Ejercicio	Qué practican
Básico	Agreguen el atributo *especialidad* al profesional y hagan que llegue hasta el JSON. Anoten por cuántos paquetes tuvieron que pasar.	Recorrido de capas, mappers, DTO
Básico	Escriban una prueba que demuestre que no se puede registrar un cliente con un teléfono de 5 dígitos.	Pruebas de dominio
Medio	Agreguen un **tiempo de descanso** de 10 minutos entre turnos del mismo profesional, modificando solo `ValidadorAgenda`.	Regla de dominio, OCP
Medio	Creen un adaptador de entrada que lea un CSV de turnos y los agende con el mismo caso de uso.	Adapter, DIP
Medio	Agreguen *excepciones* a la disponibilidad (un festivo o unas vacaciones) y respétenlas al agendar.	Evolución del modelo
Medio	Permitan que el rol USER vea solo sus turnos. Modifiquen la política y prueben los 403.	Seguridad por roles
Avanzado	Escriban una prueba con dos hilos que agenden el mismo horario del mismo profesional y comprueben que solo uno gana.	Concurrencia, bloqueo de fila
Avanzado	Agreguen paginación al listado de turnos sin tocar el dominio. ¿Qué puertos tuvieron que ampliar?	Puertos, evolución del diseño

Lista de verificación antes de entregar

- ☐ `mvn verify` termina en verde en su máquina (si falla algo, documenten qué era y cómo lo corrigieron).
- ☐ El secreto **no** aparece en ningún archivo versionado ni en capturas de pantalla.
- ☐ Probaron los cinco casos de uso con un token ADMIN y comprobaron que un token USER recibe 403.
- ☐ Probaron un cruce de horarios y vieron el 409 que indica el turno en conflicto.
- ☐ Pueden explicar, sin mirar, por qué `domain` no importa Spring.
- ☐ Pueden dibujar el recorrido de `POST /api/turnos` y nombrar la clase de cada paso.

> **Cómo seguir**
> Este mismo esquema (dominio puro, puertos, adaptadores y validación local del JWT de AuthCore) se repite en todos los grupos. Lo que aprendan aquí es la plantilla: cambia el negocio, no la arquitectura.